PROGRAM 12 12_single_linked_list.c

Singly Linked List Operations

C SOURCE CODE 12_single_linked_list.c

```c
/* Implementation of Singly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

// 1. Create Node / List
void create() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("Node created successfully\n");
}

// 2. Display List
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// 3. Search Element
void search() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key, pos = 1, found = 0;
    printf("Enter element to search : ");
    scanf("%d", &key);

```

```c
    struct Node *temp = head;
    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
            break;
        }
        temp = temp->next;
        pos++;
    }
    if (!found) {
        printf("Element %d not found in the list\n", key);
    }
}

// 4. Insert at Beginning
void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = head;
    head = newNode;
    printf("Node inserted successfully at beginning\n");
}

// 5. Insert at Middle (by position)
void insertAtMiddle() {
    int val, pos, i;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter position : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed!\n");
            return;
        }
        newNode->data = val;
        newNode->next = head;
        head = newNode;
        printf("Node inserted successfully at position %d\n", pos);
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
```

```c
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = temp->next;
    temp->next = newNode;
    printf("Node inserted successfully at position %d\n", pos);
}

// 6. Insert at End
void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("Node inserted successfully at end\n");
}

// 7. Delete at Beginning
void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

// 8. Delete at Middle (by position)
void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int pos, i;
    printf("Enter position to delete : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        struct Node *temp = head;
        head = head->next;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }

```

```c
    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *delNode = temp->next;
    temp->next = delNode->next;
    printf("Deleted element: %d\n", delNode->data);
    free(delNode);
}

// 9. Delete at End
void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    struct Node *delNode = temp->next;
    temp->next = NULL;
    printf("Deleted element: %d\n", delNode->data);
    free(delNode);
}

int main() {
    int choice;

    do {
        printf("\n- - - Linked List Menu - - -\n");
        printf("1. Create\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Insert at Beginning\n");
        printf("5. Insert at Middle\n");
        printf("6. Insert at End\n");
        printf("7. Delete at Beginning\n");
        printf("8. Delete at Middle\n");
        printf("9. Delete at End\n");
        printf("10. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                create();
                break;
            case 2:
                display();
                break;
            case 3:
                search();
                break;
            case 4:
                insertAtBeginning();
                break;
            case 5:
```

```
                insertAtMiddle();
                break;
            case 6:
                insertAtEnd();
                break;
            case 7:
                deleteAtBeginning();
                break;
            case 8:
                deleteAtMiddle();
                break;
            case 9:
                deleteAtEnd();
                break;
            case 10:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 10);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
- - - Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 5 → 10 → 20 → 30 → NULL

- - - Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Linked List Menu - - -
```

```
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 → NULL

- - - Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 13 13_doubly_linked_list.c

Doubly Linked List Operations

C SOURCE CODE 13_doubly_linked_list.c

```c
/* Implementation of Doubly Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;

// 1. Create Node / List
void create() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Node created successfully\n");
}

// 2. Display List
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// 3. Search Element
void search() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int key, pos = 1, found = 0;
```

```c
    printf("Enter element to search : ");
    scanf("%d", &key);

    struct Node *temp = head;
    while (temp ≠ NULL) {
        if (temp→data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
            break;
        }
        temp = temp→next;
        pos++;
    }
    if (!found) {
        printf("Element %d not found in the list\n", key);
    }
}

// 4. Insert at Beginning
void insertAtBeginning() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode→data = val;
    newNode→prev = NULL;
    newNode→next = head;

    if (head ≠ NULL) {
        head→prev = newNode;
    }
    head = newNode;
    printf("Node inserted successfully at beginning\n");
}

// 5. Insert at Middle (by position)
void insertAtMiddle() {
    int val, pos, i;
    printf("Enter data : ");
    scanf("%d", &val);
    printf("Enter position : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (pos == 1) {
        insertAtBeginning();
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp ≠ NULL; i++) {
        temp = temp→next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
```

```
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;

    printf("Node inserted successfully at position %d\n", pos);
}

// 6. Insert at End
void insertAtEnd() {
    int val;
    printf("Enter data : ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Node inserted successfully at end\n");
}

// 7. Delete at Beginning
void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

// 8. Delete at Middle (by position)
void deleteAtMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    int pos, i;
    printf("Enter position to delete : ");
    scanf("%d", &pos);

    if (pos < 1) {
        printf("Invalid position!\n");
```

```c
        return;
    }

    if (pos == 1) {
        deleteAtBeginning();
        return;
    }

    struct Node *temp = head;
    for (i = 1; i < pos && temp ≠ NULL; i++) {
        temp = temp→next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    if (temp→prev ≠ NULL) {
        temp→prev→next = temp→next;
    }
    if (temp→next ≠ NULL) {
        temp→next→prev = temp→prev;
    }

    printf("Deleted element: %d\n", temp→data);
    free(temp);
}

// 9. Delete at End
void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    if (head→next == NULL) {
        printf("Deleted element: %d\n", head→data);
        free(head);
        head = NULL;
        return;
    }

    while (temp→next ≠ NULL) {
        temp = temp→next;
    }

    temp→prev→next = NULL;
    printf("Deleted element: %d\n", temp→data);
    free(temp);
}

int main() {
    int choice;

    do {
        printf("\n- - - Doubly Linked List Menu - - -\n");
        printf("1. Create\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Insert at Beginning\n");
        printf("5. Insert at Middle\n");
        printf("6. Insert at End\n");
        printf("7. Delete at Beginning\n");
        printf("8. Delete at Middle\n");
        printf("9. Delete at End\n");
        printf("10. Exit\n");
        printf("Enter choice : ");
        scanf("%d", &choice);

```

```
        switch (choice) {
            case 1:
                create();
                break;
            case 2:
                display();
                break;
            case 3:
                search();
                break;
            case 4:
                insertAtBeginning();
                break;
            case 5:
                insertAtMiddle();
                break;
            case 6:
                insertAtEnd();
                break;
            case 7:
                deleteAtBeginning();
                break;
            case 8:
                deleteAtMiddle();
                break;
            case 9:
                deleteAtEnd();
                break;
            case 10:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 10);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Doubly Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Doubly Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Doubly Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Doubly Linked List Menu - - -
Enter choice : 5
```

```
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 5 ⟷ 10 ⟷ 20 ⟷ 30 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Doubly Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 10 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 14 14_stack_linked_list.c

Stack Operations Using Linked List

C SOURCE CODE 14_stack_linked_list.c

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int val) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    int val = temp->data;
    top = top->next;
    free(temp);
    return val;
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
```

```
                push(value);
                break;
            case 2:
                value = pop();
                if (value != -1) {
                    printf("Popped value: %d\n", value);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 15 15_queue_linked_list.c

Queue Operations Using Linked List

C SOURCE CODE 15_queue_linked_list.c

```c
/* Implementation of Queue using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int val) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Queue Overflow\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (front == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

int dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return -1;
    }
    struct Node *temp = front;
    int val = temp->data;
    front = front->next;

    if (front == NULL) {
        rear = NULL;
    }

    free(temp);
    return val;
}

void display() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Displaying Contents:\n");
    struct Node *temp = front;
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main() {
    int choice, value;

```

```c
    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                value = dequeue();
                if (value != -1) {
                    printf("Dequeued value: %d\n", value);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 4);

    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
```

```
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```